

ARQUITETURA E DESENVOLVIMENTO EM JAVA

FASE 1 | AULA 02 -

SINGLE RESPONSIBILITY PRINCIPLE

SUMÁRIO

O QUE VEM POR AÍ?	3
HANDS ON	4
SAIBA MAIS	5
O QUE VOCÊ VIU NESTA AULA?	11
REFERÊNCIAS	12

EMANIP

O QUE VEM POR AÍ?

Nesta aula, vamos explorar o Princípio da Responsabilidade Única (Single Responsibility Principle ou SRP), que é o primeiro dos cinco princípios SOLID da programação orientada a objetos e design de software.



HANDS ON

Nesta aula prática, apresentaremos o Princípio da Responsabilidade Única (Single Responsibility Principle), o primeiro princípio SOLID. Vamos entender a importância de dividir responsabilidades em nosso código para criar classes coesas e com propósitos claros.



SAIBA MAIS

O Princípio da Responsabilidade Única (SRP) estabelece que uma classe deve ter um, e apenas um, motivo para mudar. Em outras palavras, cada classe deve ser responsável por uma única funcionalidade ou comportamento.

Conceito de Responsabilidade Única

A ideia principal do SRP é que uma classe deve focar em uma única tarefa, especializando-se nela. Isso não apenas torna o código mais compreensível e fácil de manter, mas também melhora a coesão e reduz o acoplamento entre as partes do sistema.

Quando uma classe tem múltiplas responsabilidades, mudanças em uma parte do código podem impactar outras partes, introduzindo bugs e complexidade desnecessária.

Para ficar mais claro, vejamos um exemplo de uma classe que não aplica o princípio do SRP e outra que aplica em Java.

Neste exemplo, nós temos uma classe chamada **Employee** que viola o SRP. Note que ela possui múltiplas responsabilidades, como gerenciar informações do empregado, calcular salário e gerar relatórios.

```
public class Employee {  
    private String name;  
    private double baseSalary;  
    private int hoursWorked;  
  
    // Constructor  
    public Employee(String name, double baseSalary, int hoursWorked) {  
        this.name = name;  
        this.baseSalary = baseSalary;  
        this.hoursWorked = hoursWorked;  
    }  
}
```

```
// Getters and Setters
public String getName() {
    return name;
}

public void setName(String name) {
    this.name = name;
}

public double getBaseSalary() {
    return baseSalary;
}

public void setBaseSalary(double baseSalary) {
    this.baseSalary = baseSalary;
}

public int getHoursWorked() {
    return hoursWorked;
}

public void setHoursWorked(int hoursWorked) {
    this.hoursWorked = hoursWorked;
}

// Method to calculate salary
public double calculateSalary() {
    return baseSalary * hoursWorked;
}

// Method to generate report
public String generateReport() {
    return "Employee Report: " + name;
}
```

```
}  
}
```

Código 1 — Exemplo de uma classe que não aplica o princípio do SRP
Fonte: elaborado pelo autor (2024)

Agora, refatorando ela para que possamos aplicar o SRP, nós temos:

```
public class Employee {  
    private String name;  
    private double baseSalary;  
    private int hoursWorked;  
  
    // Constructor  
    public Employee(String name, double baseSalary, int  
hoursWorked) {  
        this.name = name;  
        this.baseSalary = baseSalary;  
        this.hoursWorked = hoursWorked;  
    }  
  
    // Getters and Setters  
    public String getName() {  
        return name;  
    }  
  
    public void setName(String name) {  
        this.name = name;  
    }  
  
    public double getBaseSalary() {  
        return baseSalary;  
    }  
}
```

```
public void setBaseSalary(double baseSalary) {  
    this.baseSalary = baseSalary;  
}  
  
public int getHoursWorked() {  
    return hoursWorked;  
}  
  
public void setHoursWorked(int hoursWorked) {  
    this.hoursWorked = hoursWorked;  
}  
}
```

Código 2 — Exemplo de uma classe que aplica o princípio do SRP
Fonte: elaborado pelo autor (2024)

E duas novas classes, chamadas `ReportGenerator.java` e `SalaryCalculator.java`, com os seguintes códigos:

ReportGenerator.java

```
public class ReportGenerator {  
    public String generateReport(Employee employee) {  
        return "Employee Report: " + employee.getName();  
    }  
}
```

Código 3 — `ReportGenerator.java`
Fonte: elaborado pelo autor (2024)

SalaryCalculator.java


```
public class SalaryCalculator {  
    public double calculateSalary(Employee employee) {  
        return employee.getBaseSalary() * employee.getHoursWorked();  
    }  
}
```

Código 4 — SalaryCalculator.java
Fonte: elaborado pelo autor (2024)

Explicando a refatoração

Antes da refatoração, a classe `Employee` estava violando o Princípio da Responsabilidade Única ao acumular múltiplas responsabilidades: gerenciar informações do empregado, calcular salário e gerar relatórios.

Isso tornava a classe difícil de manter e testar, além de aumentar a probabilidade de introduzir bugs ao modificar qualquer uma dessas responsabilidades.

Depois da refatoração:

- Classe `Employee`:
 - Responsável apenas por manter as informações do empregado, como nome, salário base e horas trabalhadas.
 - Agora essa classe está focada em sua principal responsabilidade, que é encapsular os dados do empregado.
- Classe `SalaryCalculator`:
 - Responsável por calcular o salário do empregado.
 - Ao separar essa funcionalidade, podemos facilmente modificar a lógica de cálculo de salário sem afetar outras partes do sistema.
- Classe `ReportGenerator`:
 - Responsável por gerar relatórios de empregados.
 - Ao isolar a geração de relatórios, podemos facilmente alterar o formato ou conteúdo dos relatórios sem interferir em outras funcionalidades.

Benefícios da refatoração:

- **Facilidade de manutenção:** com cada classe focada em uma única responsabilidade, torna-se mais fácil localizar e corrigir problemas.
- **Testabilidade:** classes menores e com responsabilidades claras são mais fáceis de testar isoladamente.
- **Reutilização de código:** as classes `SalaryCalculator` e `ReportGenerator` podem ser reutilizadas em diferentes contextos, aumentando a modularidade do sistema.
- **Redução de riscos:** alterações em uma classe não impactam diretamente outras partes do sistema, reduzindo o risco de bugs.

Portanto, essa refatoração não só melhora a coesão de cada classe como também facilita a manutenção e evolução do código ao longo do tempo.

Vantagens do SRP

Por fim, podemos resumir as vantagens do Princípio da Responsabilidade Única da seguinte forma:

- **Facilidade de manutenção:** classes com responsabilidades únicas são mais fáceis de entender, modificar e depurar.
- **Reutilização de código:** classes focadas em uma única responsabilidade são mais fáceis de reutilizar em diferentes contextos.
- **Testabilidade:** classes menores e mais focadas são mais simples de testar, pois cada uma pode ser isolada e verificada de maneira independente.
- **Redução de riscos:** alterações em uma classe com uma única responsabilidade têm menos probabilidade de afetar outras partes do sistema.

O QUE VOCÊ VIU NESTA AULA?

Nesta aula, exploramos em profundidade o Princípio da Responsabilidade Única (Single Responsibility Principle ou SRP), o primeiro dos cinco princípios SOLID da programação orientada a objetos e design de software.



REFERÊNCIAS

CAMPUSCODE. **S.O.L.I.D.: Princípio da Responsabilidade Única.** 2020. Disponível em: <<https://www.campuscode.com.br/conteudos/s-o-l-i-d-principio-da-responsabilidade-unica>>. Acesso em: 06 ago. 2024.

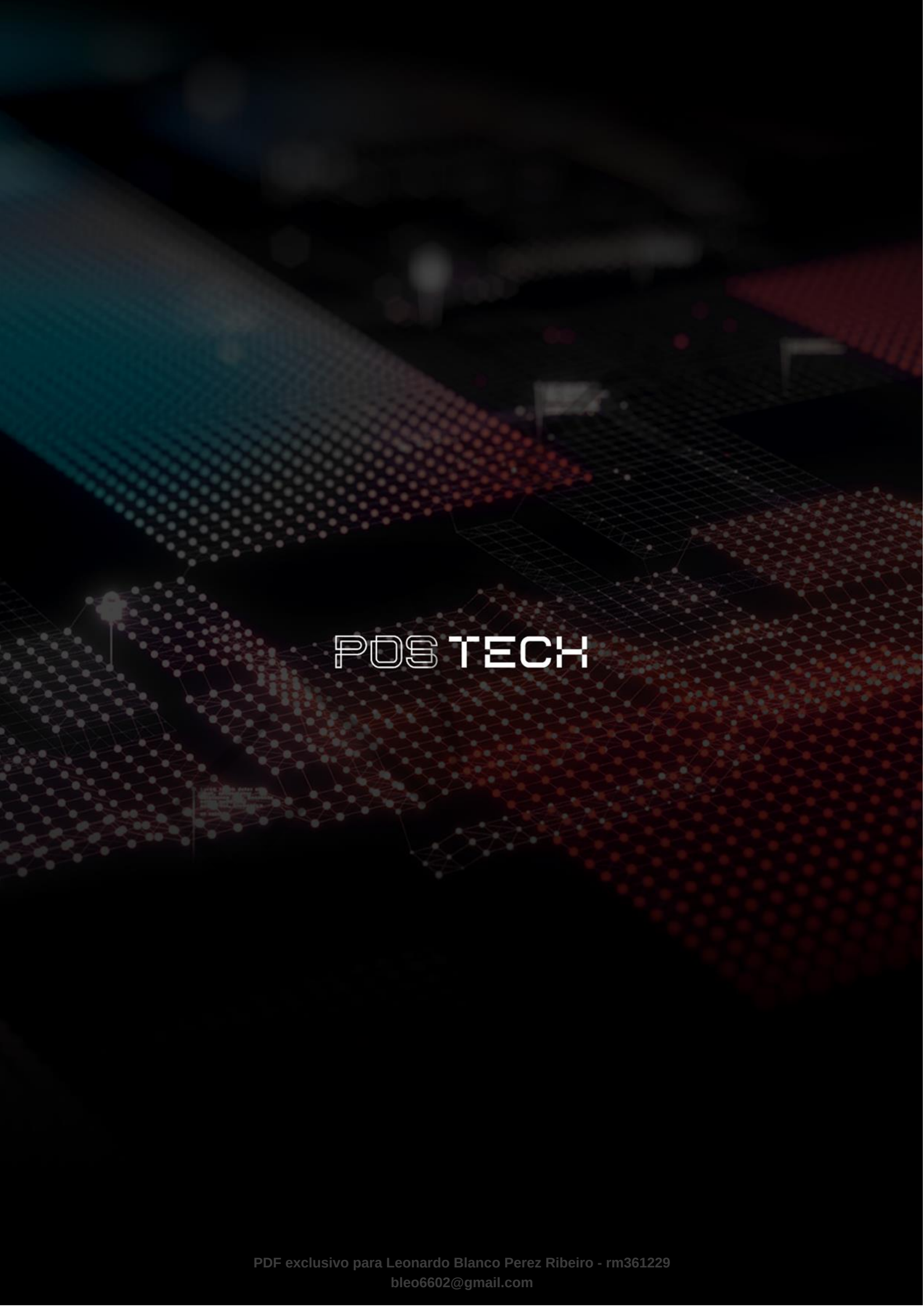
IMASTERS. **Os Princípios do SOLID — SRP Princípio da responsabilidade única.** 2019. Disponível em: <<https://medium.com/@jonesroberto/os-princ%C3%ADpios-do-solid-srp-princ%C3%ADpio-da-responsabilidade-%C3%BAunica-7897c55694fe>>. Acesso em: 06 ago. 2024.

MEDIUM. **Princípios do SOLID com TypeScript.** 2020. Disponível em: <<https://medium.com/xp-inc/princ%C3%ADpios-do-solid-com-typescript-1e585c6eeb5e>>. Acesso em: 06 ago. 2024.

PALAVRAS-CHAVE

Palavras-chave: SOLID; PRINCÍPIO DA RESPONSABILIDADE ÚNICA; SRP.

EMSE



POSTECH